rdesktop-mac

A Native Remote Desktop Protocol Client for macOS and Apple Silicon

Technical Whitepaper | Draft 0.1 | 29 October 2026

Project repository
github.com/sedwards/rdesktop-mac

Contents

Executive Summary

1. Background and Motivation

2. Project Objectives

3. Architecture Overview

4. Native macOS Integration

5. Command-Line and Application-Bundle Workflows

6. Transport Security and Authentication

7. Apple Silicon and macOS Packaging

8. Open-Source Model and Licensing

9. Current Repository Evidence and Qualification

10. Validation and Testing Strategy

11. Roadmap

12. Conclusion

Appendix A — Repository and Build References

Appendix B — Suggested Project Positioning

Executive Summary

rdesktop-mac is an open-source effort to bring the established rdesktop Remote Desktop Protocol (RDP) client architecture to modern macOS. Its purpose is to provide Mac users and administrators with a native-feeling RDP client that can be driven from the command line while also supporting a macOS application bundle and native connection interface.

The project combines the portability and command-line heritage of the original C-based rdesktop codebase with macOS-specific Objective-C and Cocoa/AppKit integration. The repository includes native display and input code, CoreGraphics rendering, macOS application packaging, and a build path that can link a locally built OpenSSL 3.x implementation. The project owner also describes the codebase as incorporating selected FreeRDP components; the precise file-level provenance and licensing of those components should be documented as part of release engineering.

This paper describes the project's goals, architecture, implementation approach, current evidence visible in the public repository, security considerations, and a practical roadmap. It intentionally distinguishes implemented repository elements from capabilities that still require validation. It does not claim independent security certification, comprehensive protocol conformance, or measured performance superiority over other RDP clients.

1. Background and Motivation

Remote Desktop Protocol is widely used to access Windows desktops and Remote Desktop Services from other operating systems. On macOS, users have several client choices, but command-line workflows, native operating-system integration, dependency management, and control over the client's implementation can be important to developers, system administrators, and users who prefer open-source software.

The original rdesktop project established a compact, C-based command-line approach to RDP. The rdesktop-mac project takes that foundation and adapts its user-interface and platform-facing components for macOS rather than treating macOS as simply another Unix-like X11 target. The repository includes Objective-C source files, an Xcode project, macOS-specific build instructions, and a native application-bundle target.

The motivation is practical: provide a scriptable RDP client that fits into macOS workflows, builds with Apple's developer toolchain, and can be packaged as a recognizable .app application. The project is particularly relevant to Apple Silicon Macs, where a native arm64 build can avoid relying on an x86 virtualization or translation layer. The repository's build process should nevertheless be validated on each supported macOS and processor combination before publishing a formal compatibility matrix.

2. Project Objectives

The project's principal objectives are:

- Preserve a familiar rdesktop-style command-line interface and workflow.
- Provide native macOS display, window, keyboard, and pointer integration.
- Support an application bundle that can be launched through Finder or the open command.
- Keep the core client implementation accessible to developers through an open-source codebase.
- Use established cryptographic libraries for secure transport rather than inventing cryptographic primitives.
- Improve maintainability through reproducible builds, explicit dependency provenance, protocol testing, and security review.

The intended audience includes macOS users who administer Windows systems, developers who need scriptable remote sessions, IT teams that prefer open-source tooling, and contributors interested in native macOS networking and graphics integration.

3. Architecture Overview

At a high level, rdesktop-mac separates the RDP client core from macOS-specific platform services. The inherited C code handles protocol/session functions and remote desktop data. The macOS layer translates between that core and native windowing, graphics, and input facilities.

The public repository contains core RDP source modules (including protocol, channel, bitmap, cache, and transport files), Objective-C implementation files, a macOS Xcode project, resource assets, and build scripts. This structure provides a path to retain the protocol engine while replacing or adapting platform-dependent services.

The project owner reports that selected FreeRDP code has also been incorporated. The public files reviewed for this draft do not yet provide a clear component-by-component attribution map. A release-quality architecture document should identify each imported or adapted component, its upstream revision, local modifications, and applicable license notices. This is important both for maintainability and for satisfying third-party license obligations.

4. Native macOS Integration

The repository's macOS user-interface implementation is written in Objective-C and uses Cocoa/AppKit. The native window and view layer provides the place where remote desktop updates are presented and where local user input is captured. This avoids requiring an X11 server merely to display the remote session.

CoreGraphics is used for the display backing store and bitmap presentation. The macOS build notes document an explicit pixel-layout mapping for RDP BGRX/BGRA data and CoreGraphics bitmap contexts. They also document scaling mouse coordinates from view-relative points into the remote session's pixel dimensions, which is important when a window is resized or displayed on a Retina screen.

The macOS layer uses an input-event queue protected by a mutex to move mouse and keyboard events between the UI and the RDP processing path. The build notes also describe redraw handling intended to reduce delayed or incomplete repainting. These are meaningful platform adaptations: correct pixel formats, coordinate transforms, event synchronization, and redraw behavior are central to a usable remote desktop client.

The repository's build guide lists remaining display work, including flicker and redraw artifacts observed on an M4 system. These items should remain visible in the project's issue tracker and test matrix rather than being described as fully resolved.

5. Command-Line and Application-Bundle Workflows

The project is intended to serve both terminal-oriented and desktop-oriented users. The build guide describes creating rdesktop-mac.app and launching the connection interface with `open rdesktop-mac.app`. It also documents invoking the executable inside the bundle directly with command-line arguments, for example a username, password, and host address.

This dual workflow is a useful differentiator in project design. A command-line interface can support shell scripts, repeatable connection profiles, remote administration routines, and automation. The .app bundle provides a familiar macOS launch path and a native connection dialog. The two entry points should share the same connection engine and option semantics so that behavior remains consistent.

Before publishing a stable CLI reference, the project should maintain a generated or tested `--help` output, document every supported option, and provide examples for authentication, display size, clipboard, audio, drive redirection, and certificate handling only where those features are actually implemented and tested.

6. Transport Security and Authentication

RDP sessions carry credentials and remote desktop data, so transport security is a foundational requirement. The repository includes an OpenSSL migration specification describing the transition from GnuTLS to OpenSSL, including TLS session setup, certificate verification, encrypted reads and writes, and cleanup. The macOS build guide describes a bundled static OpenSSL 3.x path intended to reduce runtime dependency requirements and produce a self-contained application bundle.

The build guide also describes a native Secure Transport pathway using Apple's Security framework. However, inspection of the current `macssl.c` source shows placeholder implementations for several legacy cryptographic operations, including SHA-1, MD5, RC4, and RSA-related routines. Those functions must not be represented as a complete or safe cryptographic implementation. The native Secure Transport pathway should be considered experimental or disabled until every required operation is correctly implemented and independently tested. The bundled OpenSSL path should be the documented default only after its actual build and runtime behavior have been verified.

The repository's build notes describe a static authentication component (`libnlav2.a`) and a nested OpenSSL source tree. The project should document the exact NLA/CredSSP implementation, upstream provenance, supported authentication modes, and interoperability results. Authentication support should not be inferred solely from the presence of a dependency or a build target.

Security recommendations for users: do not expose an RDP service or client-side credentials unnecessarily; use trusted networks, VPNs, or Remote Desktop Gateways where appropriate; validate server certificates; avoid disabling certificate checks as a routine workaround; and do not use a build that includes placeholder cryptographic code for sensitive sessions.

7. Apple Silicon and macOS Packaging

The project's native Objective-C/Cocoa implementation and macOS build system provide a foundation for Apple Silicon support. The build script invokes the local toolchain and creates a macOS application bundle. The build documentation describes a local static OpenSSL build, compilation of the authentication library, compilation of the client, and bundle assembly. It also documents generation of a macOS ICNS icon using native system utilities and AppKit APIs.

A native build on Apple Silicon can provide a direct arm64 execution path and avoid the overhead and compatibility limitations of running an Intel-only client under translation. This is an architectural goal, not a substitute for published test evidence. The project should verify the executable architecture with `file` and `lipo -info`, record the minimum macOS deployment target, and test clean builds on representative M-series machines.

For distribution beyond developer machines, the project should consider code signing, notarization, hardened runtime compatibility, a clear update process, and a documented policy for bundled OpenSSL updates. The current repository build guide describes bundle assembly but does not by itself establish that distributed builds are signed, notarized, or independently security-reviewed.

8. Open-Source Model and Licensing

The repository identifies its license as GNU General Public License version 3 (GPL-3.0). This makes the project available for use, study, modification, and redistribution subject to the license's conditions. "Free to use" should therefore be communicated as free/open-source software, not as public-domain software or software without obligations.

Because the project is derived from rdesktop and the project owner reports incorporating selected FreeRDP components, the repository should maintain complete copyright notices, license texts, source attribution, and a third-party software inventory. The exact licensing compatibility and notice requirements must be checked against the actual code included in the distribution. The project should not imply that all upstream code has the same license

without a file-level review.

A clear contribution policy, security reporting contact, issue triage process, and release/versioning policy would help establish a sustainable community project.

9. Current Repository Evidence and Qualification

The repository reviewed for this draft contains a macOS build guide, an automated build script, an Xcode project, Objective-C macOS UI files, CoreGraphics rendering code, an OpenSSL migration specification, and a GPL-3.0 license declaration. The build guide documents the application-bundle workflow and several implementation details around pixel format, Retina coordinate scaling, and redraw behavior.

The repository's top-level README still contains inherited rdesktop descriptions and warnings about the maintenance state and security history of the parent project. It does not yet explain the macOS port's architecture, the project's own feature status, or its security posture in sufficient detail. Replacing that inherited README with current project-specific documentation is a high-priority improvement.

The build guide's errata list identifies M4 redraw flicker/artifacts and other cleanup items. The current public documentation does not provide a comprehensive tested-feature matrix, performance benchmarks, protocol-version matrix, or formal security audit. This whitepaper therefore describes the project's architecture and direction without claiming those unverified results.

The project owner's description of FreeRDP-derived components is included as project-provided context. A source-level provenance audit should be completed before the whitepaper is treated as a definitive bill of materials.

10. Validation and Testing Strategy

A credible release process should include repeatable tests across the following areas:

- Build validation: clean builds using the documented script; supported Xcode/Command Line Tools versions; arm64 architecture checks; dependency and linker inspection.
- Connection interoperability: Windows client/server versions and common RDP server implementations such as Windows Remote Desktop Services and xrdp, with exact versions recorded.
- Authentication: standard credentials and NLA/CredSSP flows where supported; invalid credentials; expired or untrusted certificates; hostname mismatch; reconnect and disconnect.
- Graphics: multiple resolutions, Retina scaling, window resize, color accuracy, high-motion content, redraw stress, and long-running sessions.
- Input: keyboard layouts, modifier keys, special keys, mouse buttons, wheel events, pointer scaling, and rapid input.
- Channels: clipboard, audio, drive/device redirection, and smart-card behavior only where implemented.
- Reliability: network interruption, server restart, sleep/wake, repeated connect/disconnect, memory usage, and thread-safety stress.
- Security: dependency scanning, compiler warnings, AddressSanitizer, fuzzing of parsers where practical, certificate-validation tests, and independent review of cryptographic and authentication code.

Performance claims should be backed by repeatable measurements: CPU use, memory use, frame rate, input latency, bandwidth, and reconnect time. Results should identify the Mac model, macOS version, server type, network conditions, session resolution, and test workload.

11. Roadmap

Near-term priorities:

1. Replace the inherited README with project-specific documentation and a tested feature matrix.
2. Complete a file-level FreeRDP and third-party code attribution inventory.
3. Make the OpenSSL-backed TLS path the only supported secure path until the native Security.framework implementation is complete and reviewed.
4. Resolve known M4 redraw flicker and artifact issues; add regression tests for Retina scaling and pixel formats.
5. Publish reproducible build instructions, minimum macOS version, supported CPU architectures, and a dependency manifest.
6. Document CLI options and provide practical connection examples.
7. Establish a release process with signed/notarized artifacts where appropriate.

Medium-term priorities:

1. Expand interoperability testing across Windows RDP and xrdp versions.
2. Improve diagnostics, structured logging, and recoverable connection errors.
3. Add automated tests for reconnects, clipboard, audio, and device channels as those features mature.
4. Benchmark the client on representative M-series hardware and compare results using reproducible workloads.
5. Establish contributor guidance, issue templates, and a security disclosure process.

12. Conclusion

rdesktop-mac is an effort to make the rdesktop command-line approach useful in the modern macOS environment by combining a mature C-based RDP foundation with native Cocoa, Objective-C, and CoreGraphics integration. Its dual command-line and application-bundle workflows are intended to make it useful to both terminal-driven administrators and desktop users. The repository's build system and OpenSSL packaging work provide a foundation for a native macOS distribution, including Apple Silicon targets.

The project's strongest public story is its focus: a native macOS RDP client that remains scriptable and open source. Its next stage should emphasize clear documentation, transparent component attribution, tested Apple Silicon support, dependable graphics behavior, and a security implementation that can withstand careful review. With those elements in place, rdesktop-mac can offer the community a maintainable alternative for remote desktop access on modern Macs.

Appendix A — Repository and Build References

Primary repository: https://github.com/sedwards/rdesktop-mac

macOS build guide: https://github.com/sedwards/rdesktop-mac/blob/master/BUILD_MAC.md

OpenSSL migration specification:
https://github.com/sedwards/rdesktop-mac/blob/master/OPENSSL_MIGRATION_SPEC.md

macOS display implementation: https://github.com/sedwards/rdesktop-mac/blob/master/macwin.m

Native TLS implementation: https://github.com/sedwards/rdesktop-mac/blob/master/macssl.c

FreeRDP upstream project: https://github.com/FreeRDP/FreeRDP

This draft is based on the public repository contents available on 29 October 2026 and on project-owner context supplied with the request. Repository contents and implementation status may change. The paper should be updated alongside tagged releases.

Appendix B — Suggested Project Positioning

Short description: “rdesktop-mac is an open-source, native macOS RDP client that brings the rdesktop command-line workflow to modern Macs, with Cocoa-based desktop integration and an Apple Silicon-oriented build path.”

Suggested tagline: “Remote Desktop for macOS. Native. Scriptable. Open Source.”

Claims to reserve until measured or independently verified: “faster than Microsoft Remote Desktop,” “more secure than FreeRDP,” “full RDP compatibility,” “production-ready NLA,” “universal Apple Silicon support,” or “zero runtime dependencies.”